

The Chaos Language Algorithm: An Implementation-Oriented Specification

Ben Goertzel w/ GPT-5.5-Pro

Abstract

The Chaos Language Algorithm (CLA) converts a continuous chaotic trajectory into a symbolic string and then induces a compact grammar for that string. The crucial distinction in this specification is between *chunks* and *meta-symbols*. A chunk is a repeated sequential block, represented by a nonterminal rule such as $A \rightarrow \beta$. A meta-symbol is not a block. A meta-symbol is a grammatical category: a set of tokens that are substitutable in similar contexts. The algorithm therefore searches over two orthogonal forms of compression: sequential compression by chunking, and categorical compression by grouping contextually interchangeable tokens.

1 Purpose

Given a scalar or vector time series produced by a deterministic or stochastic dynamical system, the goal is to infer a grammar G^* that compactly describes the observed symbolic trajectory and generalizes it in a controlled way. The induced grammar is selected by a two-part minimum description length (MDL) objective:

$$G^* = \arg \min_G [L(G) + L(S \mid G)], \quad (1)$$

where $L(G)$ is the code length of the grammar and $L(S \mid G)$ is the code length of the observed string given that grammar. All accepted operations must preserve exact reconstruction of the observed string, unless an explicit noise or exception model is added.

2 Objects and Notation

U The observed continuous trajectory. For a scalar series, $U = (u_1, \dots, u_T)$.

d, τ The embedding dimension and delay used for time-delay reconstruction.

x_t The reconstructed state vector

$$x_t = (u_t, u_{t-\tau}, \dots, u_{t-(d-1)\tau}) \in \mathbb{R}^d. \quad (2)$$

$\mathcal{P}$ A finite partition of reconstructed state space:

$$\mathcal{P} = \{C_1, \dots, C_k\}. \quad (3)$$

Σ_0 The base alphabet, with one base symbol for each partition cell:

$$\Sigma_0 = \{\sigma_1, \dots, \sigma_k\}. \quad (4)$$

ϕ The cell-labeling map $\phi : C_j \mapsto \sigma_j$.

S_0 The raw symbolic trajectory:

$$S_0 = s_1 s_2 \cdots s_n, \quad s_i \in \Sigma_0. \quad (5)$$

V The working vocabulary. During induction,

$$V = \Sigma_0 \cup \mathcal{N} \cup \mathcal{M}, \quad (6)$$

where $\mathcal{N}$ is the set of nonterminals created by chunking, and $\mathcal{M}$ is the set of meta-symbols created by categorization.

$A \in \mathcal{N}$ A nonterminal naming a repeated sequence. A chunk rule has the form

$$A \rightarrow \beta, \quad \beta \in V^+. \quad (7)$$

A nonterminal abbreviates a string. It is syntagmatic: it captures “and-then” structure.

$M \in \mathcal{M}$ A meta-symbol. It has an extension

$$\text{Ext}(M) \subseteq V. \quad (8)$$

A meta-symbol abbreviates a choice among substitutable tokens. It is paradigmatic: it captures “one-of-these” structure. It is a category, not a sequential block.

G The current grammar, containing chunk rules, category definitions, and any start or parse rules needed to reproduce the observed string.

3 Semantic Distinction: Chunks Versus Meta-Symbols

The algorithm uses two different representational moves.

Chunking. If a sequence $\beta = v_1 \cdots v_m$ recurs often enough, introduce a nonterminal

$$A \rightarrow v_1 \cdots v_m \quad (9)$$

and rewrite selected non-overlapping occurrences of β as A . This reduces repeated sequential material.

Categorization. If tokens $v_1, \dots, v_m$ occur in similar left and right contexts, introduce a meta-symbol

$$M = \{v_1, \dots, v_m\}. \quad (10)$$

Then a rule or parse expression containing M abbreviates alternatives. For example,

$$A \rightarrow aMb \quad (11)$$

stands for the family

$$\{A \rightarrow avb : v \in \text{Ext}(M)\}. \quad (12)$$

To preserve exact reconstruction, each occurrence of M in the parse must either point to the observed member $v \in \text{Ext}(M)$ or be encoded with a conditional member code $-\log_2 p(v \mid M)$.

4 Phase 1: Symbolic Dynamics

4.1 Embedding

If the input is scalar, reconstruct the attractor with time-delay embedding. Practical implementations may choose τ by the first minimum of average mutual information and d by false nearest neighbors, but CLA itself only requires a fixed embedding map.

4.2 Partitioning

Partition the reconstructed state space into cells $\mathcal{P} = \{C_1, \dots, C_k\}$. The partition may be a grid, clustering partition, learned decision tree, Voronoi tessellation, or a problem-specific symbolic dynamics partition. A generating partition is ideal, but the later grammar induction stage is designed to compensate for crude initial partitions.

4.3 Labeling

Assign each point x_t the symbol of the cell it occupies:

$$s_t = \phi(x_t). \quad (13)$$

The result is the base string $S_0 = s_1 \cdots s_n$ over Σ_0 .

5 Phase 2: Joint Chunk and Category Induction

Phase 2 searches over two classes of grammar edits.

5.1 Chunk Edit

A chunk edit is specified by a repeated block $\beta \in V^+$ and a new nonterminal A . It adds a rule $A \rightarrow \beta$ and rewrites selected non-overlapping occurrences of β as A . A chunk edit is valid only if it preserves exact expansion back to the previous stream.

A practical implementation should enforce two Re-Pair/SEQUITUR-like invariants:

- (i) **Rule utility:** every chunk nonterminal is used at least twice, unless it is the start symbol.
- (ii) **No dead rules:** if a rule's use count drops below two, inline it and remove it.

5.2 Category Edit

A category edit is specified by a set $C \subseteq V$, usually obtained by clustering tokens with similar context distributions. It introduces a meta-symbol M with $\text{Ext}(M) = C$.

For each token $v \in V$, define a context vector $c(v)$. The simplest usable context vector uses immediate left and right neighbors:

$$c(v) = (P(\ell \mid v), P(r \mid v))_{\ell, r \in V \cup \{\perp, \top\}}, \quad (14)$$

where $\perp$ and $\top$ mark beginning and end of stream. Wider context windows may be used. Two tokens are categorically similar when their context distributions are close, for example when

$$\text{JS}(c(v), c(w)) \leq \theta. \quad (15)$$

A category edit is accepted only when the resulting grammar and parse reduce the MDL objective. This prevents arbitrary clustering from inflating the grammar.

5.3 Renormalization

After an accepted chunk or category edit, the current stream, parse, rule uses, and context statistics are updated. The next round operates on the new vocabulary. Thus a later chunk may contain earlier chunks, and a later category may group base symbols, chunk nonterminals, or previously created category-bearing parse atoms when the implementation allows it.

6 Phase 3: MDL Selection

The induced grammar is selected by minimizing a computable code length. A recommended exact-reconstruction objective is

$$L(G, S) = L_{\text{model}}(G) + L_{\text{data}}(S \mid G). \quad (16)$$

6.1 Model Cost

The model cost includes the bits needed to encode all rule definitions and all category extensions. One simple implementation is:

$$L_{\text{model}}(G) = \sum_{A \rightarrow \beta \in R} [c_R + \log_2^* |\beta| + |\beta| \log_2 |V|] + \sum_{M \in \mathcal{M}} \left[c_M + \log_2 \binom{|V|}{|\text{Ext}(M)|} \right], \quad (17)$$

where c_R and c_M are fixed overhead constants and $\log_2^*$ is a universal integer code. Other prefix-code choices are acceptable if they are used consistently.

6.2 Data Cost

The data cost is the number of bits needed to reproduce the observed stream from the grammar. For an exact parse stream $P = e_1 \cdots e_m$, one simple coding scheme is

$$L_{\text{data}}(S \mid G) = \sum_{i=1}^m L(e_i), \quad (18)$$

where terminals and nonterminals use a token code, while a category occurrence $M[v]$ uses a code for M plus a member code:

$$L(M[v]) = L(M) - \log_2 p(v \mid M). \quad (19)$$

The member distribution $p(v \mid M)$ may be uniform, empirical, or smoothed empirical. Uniform coding is simpler; empirical coding is often more compressive but must include the cost of encoding its parameters.

6.3 Acceptance Rule

A proposed edit q is accepted when

$$\Delta L(q) = L(G_q, S) - L(G, S) < 0. \quad (20)$$

A greedy implementation accepts the best negative- ΔL edit at each iteration. A stronger implementation may use beam search, simulated annealing, or reversible-jump search over edit sequences.

7 Pseudocode

Algorithm 1 Chaos Language Algorithm (CLA)

Require: Trajectory U ; embedding parameters d, τ ; partition routine Π ; maximum chunk length $n_{\max}$; context width w ; category threshold θ ; maximum iterations $I_{\max}$.

Ensure: Grammar G^* , parse P^* , compressed stream S^* .

```

1:  $X \leftarrow \text{EMBED}(U, d, \tau)$ 
2:  $\mathcal{P} \leftarrow \text{PARTITION}(X, \Pi)$ 
3:  $(S, \Sigma_0) \leftarrow \text{LABELTRAJECTORY}(X, \mathcal{P})$ 
4:  $G \leftarrow \text{INITIALIZEGRAMMAR}(\Sigma_0)$ 
5:  $P \leftarrow S$ 
6:  $L \leftarrow \text{MDL}(G, P)$ 
7: for  $i = 1$  to  $I_{\max}$  do
8:    $Q_c \leftarrow \text{CHUNKPROPOSALS}(S, G, n_{\max})$ 
9:    $Q_m \leftarrow \text{CATEGORYPROPOSALS}(S, G, w, \theta)$ 
10:   $Q \leftarrow Q_c \cup Q_m$ 
11:   $(q^*, L^*) \leftarrow \text{BESTACCEPTEDEDIT}(Q, G, S, P, L)$ 
12:  if  $q^* = \emptyset$  then
13:    break
14:  end if
15:   $(G, S, P) \leftarrow \text{APPLYEDIT}(q^*, G, S, P)$ 
16:   $(G, S, P) \leftarrow \text{PRUNEDeadRULES}(G, S, P)$ 
17:   $L \leftarrow L^*$ 
18: end for
19: return  $(G, S, P)$ 

```

Algorithm 2 ChunkProposals

Require: Current stream S ; grammar G ; maximum length $n_{\max}$.

Ensure: Set of candidate chunk edits.

```

1:  $Q \leftarrow \emptyset$ 
2: for  $n = 2$  to  $n_{\max}$  do
3:    $C \leftarrow \text{COUNTNGRAMS}(S, n)$ 
4:   for all  $\beta$  with  $C[\beta] \geq 2$  do
5:      $O \leftarrow \text{NONOVERLAPPINGOCCURRENCES}(S, \beta)$ 
6:     if  $|O| \geq 2$  then
7:        $A \leftarrow \text{FRESHNONTERMINAL}$ 
8:        $q \leftarrow \text{MAKECHUNKEDIT}(A, \beta, O)$ 
9:        $Q \leftarrow Q \cup \{q\}$ 
10:    end if
11:  end for
12: end for
13: return  $Q$ 

```

Algorithm 3 CategoryProposals

Require: Current stream S ; grammar G ; context width w ; threshold θ .

Ensure: Set of candidate category edits.

```
1:  $Q \leftarrow \emptyset$ 
2:  $V \leftarrow \text{WORKINGVOCABULARY}(G, S)$ 
3: for all  $v \in V$  do
4:    $h_v \leftarrow \text{CONTEXTHISTOGRAM}(S, v, w)$ 
5:    $c_v \leftarrow \text{SMOOTHANDNORMALIZE}(h_v)$ 
6: end for
7:  $\mathcal{C} \leftarrow \text{CLUSTERBYCONTEXTSIMILARITY}(\{c_v : v \in V\}, \text{JS}, \theta)$ 
8: for all  $C \in \mathcal{C}$  with  $|C| \geq 2$  do
9:    $M \leftarrow \text{FRESHMETASYMBOL}$ 
10:   $F \leftarrow \text{FINDFRAMES}(S, C, w)$   $\triangleright$  frames are contexts where members of  $C$  appear
11:  if  $\text{HASUSEFULFRAME}(F)$  then
12:     $q \leftarrow \text{MAKECATEGORYEDIT}(M, C, F)$ 
13:     $Q \leftarrow Q \cup \{q\}$ 
14:  end if
15: end for
16: return  $Q$ 
```

Algorithm 4 BestAcceptedEdit

Require: Candidate edits Q ; grammar G ; stream S ; parse P ; current length L .

Ensure: Best edit with negative MDL change, or $\emptyset$.

```
1:  $q^* \leftarrow \emptyset$ 
2:  $L^* \leftarrow L$ 
3: for all  $q \in Q$  do
4:    $(\hat{G}, \hat{S}, \hat{P}) \leftarrow \text{PREVIEWEDIT}(q, G, S, P)$ 
5:   if not  $\text{EXACTRECONSTRUCTS}(\hat{G}, \hat{P}, S_0)$  then
6:     continue
7:   end if
8:    $\hat{L} \leftarrow \text{MDL}(\hat{G}, \hat{P})$ 
9:   if  $\hat{L} < L^*$  then
10:     $q^* \leftarrow q$ 
11:     $L^* \leftarrow \hat{L}$ 
12:   end if
13: end for
14: return  $(q^*, L^*)$ 
```

8 Output

The final output is a tuple

$$(G^*, S^*, P^*) \tag{21}$$

where G^* is the induced grammar, S^* is the compressed working stream, and P^* is the exact parse or derivation data required to reconstruct the original symbolic trajectory S_0 .

A useful textual output format is:

Base alphabet:

sigma_1, sigma_2, ..., sigma_k

Chunk rules:

A_1 -> sigma_2 sigma_5 sigma_1

A_2 -> A_1 sigma_3

Meta-symbols:

M_1 = {sigma_4, sigma_7, A_1}

M_2 = {sigma_2, sigma_9}

Grammar templates:

B_1 -> sigma_1 M_1 sigma_3

Parse stream:

A_2 M_1[sigma_7] A_2 M_1[A_1] ...

The bracketed member in $M[v]$ is not optional if exact reconstruction is required. It records which member of the category actually occurred.

9 Workflow Diagram

[Continuous trajectory U]

|

| Phase 1: embed, partition, label

v

[Raw symbolic string S_0 over Sigma_0]

|

| Phase 2a: chunk repeated sequences -> nonterminals A -> beta

| Phase 2b: categorize substitutable tokens -> meta-symbols M={v_i}

| Phase 2c: renormalize and repeat under MDL

v

[Hierarchical grammar over symbols, chunks, and categories]

|

| Phase 3: select G minimizing L(G)+L(S|G)

v

[Predictive grammar G^* plus exact parse data]

A Appendix: Guidance for Coding Agents

A.1 Non-Negotiable Semantic Constraints

A coding agent implementing CLA must preserve the following distinctions.

C1. A chunk is a sequence abbreviation. Implement it as a nonterminal rule $A \rightarrow \beta$.

C2. A meta-symbol is a category. Implement it as a set-valued object M with $\text{Ext}(M) \subseteq V$.

- C3.** Do not implement a meta-symbol as a compressed n-gram. The edit that replaces *abc* by a new token is a chunk edit, not a category edit.
- C4.** A category occurrence must preserve the identity of the observed member. Store it as $M[v]$ or store an equivalent side table mapping parse positions to members.
- C5.** Every accepted edit must reduce the same MDL objective used by the rest of the program. Do not use frequency alone as an acceptance rule.
- C6.** The implementation must be able to expand G and P back to the original raw string S_0 after every accepted edit.

A.2 Recommended Data Model

The following type sketch is intentionally language-neutral.

```

TokenId      = integer
RuleId       = integer
CategoryId   = integer
Position     = integer

Atom =
  Terminal(TokenId)
| Nonterminal(RuleId)
| CategoryUse(CategoryId, chosen_member=TokenId)

Rule = {
  id: RuleId,
  rhs: list[Atom],
  use_count: integer
}

Category = {
  id: CategoryId,
  members: set[TokenId],
  member_code: dict[TokenId, float],
  context_signature: optional vector
}

Grammar = {
  base_alphabet: set[TokenId],
  rules: dict[RuleId, Rule],
  categories: dict[CategoryId, Category],
  start: optional RuleId
}

State = {
  original_stream: list[TokenId],
  current_stream: list[Atom],

```

```

    parse: list[Atom],
    grammar: Grammar,
    mdl_score: float
}

Proposal = {
    kind: "chunk" or "category",
    payload: kind-specific data,
    predicted_delta: optional float,
    inverse_patch: optional data
}

```

A.3 Minimal Milestone Plan

A robust implementation can be built in five milestones.

- M1. Symbolization only.** Convert a trajectory to S_0 using a fixed partition. Unit test that the same point always maps to the same symbol.
- M2. Chunk-only grammar.** Implement repeated n-gram proposals, non-overlapping replacement, rule expansion, rule use counts, and exact reconstruction.
- M3. MDL scoring.** Replace frequency thresholds with a bit-based objective. Unit test that an edit is rejected when its model overhead exceeds its data savings.
- M4. Hard category induction.** Compute context histograms, cluster tokens by context similarity, create disjoint meta-symbols, and store category occurrences as $M[v]$.
- M5. Unified search.** Put chunk and category proposals into the same priority queue and accept the proposal with the largest negative ΔL .

A.4 Context Distribution Details

For a first implementation, use a context width of $w = 1$. For every occurrence of token v at position i , count

$$(s_{i-1}, \square) \quad \text{and} \quad (\square, s_{i+1}), \quad (22)$$

using boundary symbols at the beginning and end of the stream. Smooth counts with a small constant $\alpha > 0$ before normalizing. Compare context vectors with Jensen-Shannon divergence or cosine distance.

A category candidate C should usually satisfy:

- (i) $|C| \geq 2$;
- (ii) each member has a minimum occurrence count;
- (iii) average pairwise context distance is below threshold θ ;
- (iv) the candidate reduces $L(G, S)$ after member choices are encoded.

A.5 Replacement Semantics

Use different replacement semantics for chunks and categories.

Chunk replacement. A chunk replacement rewrites contiguous substrings:

$a\ b\ c\ a\ b\ c\ d \rightarrow A\ A\ d$ with rule $A \rightarrow a\ b\ c$

The parse expands A back to abc .

Category replacement. A category replacement rewrites choices inside equivalent frames:

$a\ x\ b\ a\ y\ b\ a\ x\ b \rightarrow a\ M[x]\ b\ a\ M[y]\ b\ a\ M[x]\ b$
 $M = \{x, y\}$

The parse expands $M[x]$ to x and $M[y]$ to y . The grammar may now represent the generalized frame aMb .

A.6 Acceptance Tests

A coding agent should include at least the following tests.

- T1. Exact expansion.** After every accepted edit, expanding the parse returns exactly S_0 .
- T2. Chunk identity.** If $A \rightarrow abc$, then every A expands to abc and never to a set of alternatives.
- T3. Category identity.** If $M = \{x, y\}$, then M is not expanded as xy or yx ; it expands to exactly one chosen member at each occurrence.
- T4. Frame generalization.** From
 $a\ x\ b\ a\ y\ b\ a\ x\ b$
the system may propose $M = \{x, y\}$ in the frame $a.b$.
- T5. MDL rejection.** A proposed category containing unrelated rare symbols must be rejected when its extension cost is larger than its compression savings.
- T6. Rule pruning.** If a chunk rule's use count falls below two, it is inlined or removed.
- T7. Determinism.** With the same random seed and input, the same sequence of accepted edits is produced.

A.7 Common Failure Modes

- F1. Conflating chunks and categories.** This is the main error to avoid. Repeated string abc implies a possible chunk. Similar contexts for x and y imply a possible category.
- F2. Accepting by frequency only.** A frequent block or cluster is not useful unless it reduces total description length.
- F3. Losing exact reconstruction.** If category occurrences do not store chosen members, the algorithm cannot reconstruct the observed trajectory.
- F4. Overlapping category ambiguity.** Start with hard, disjoint categories. Allow overlapping categories only after the parse stores which category sense is used at each position.
- F5. Unstable indices after rewriting.** Chunk replacements change stream length. Recompute or incrementally update occurrence indices rather than reusing stale positions.

A.8 Recommended Defaults

partition:	equal-frequency bins or k-means cells
n_max:	4 to 8 for first implementation
min_chunk_uses:	2
context_width:	1 initially, then 2
smoothing_alpha:	0.1 or 1.0
category_distance:	Jensen-Shannon divergence
category_policy:	hard/disjoint initially
search:	greedy best negative delta-L
reconstruction:	exact, no noise model at first
random_seed:	fixed in tests

A.9 Instruction Block for an Automated Coding Agent

The following block may be pasted directly into an implementation task.

Implement the Chaos Language Algorithm from the attached specification. Preserve the distinction between chunks and meta-symbols. A chunk is a nonterminal rule $A \rightarrow \text{beta}$ where beta is a contiguous sequence. A meta-symbol M is a category with $\text{Ext}(M) = \{v_1, \dots, v_k\}$; each occurrence must record the chosen member as $M[v]$. Implement exact reconstruction checks after every accepted edit. Use a computable MDL objective, not raw frequency, as the acceptance criterion. Build in this order: symbolizer, chunk proposals, expansion/reconstruction tests, MDL scoring, context histograms, hard category proposals, then unified greedy search. Include unit tests for exact expansion, category member recovery, rule pruning, and rejection of MDL-worsening edits.